

Laporan Penugasan: Pembangunan Aplikasi Monolitik Stateful dan Deployment pada AWS EC2

Nama: Ifham Syafwan Fikri
NIM: 24/545184/PA/23161

1. Pendahuluan

Penugasan ini saya kerjakan untuk memenuhi instruksi utama mata kuliah Pengembangan Perangkat Lunak Scalable, yaitu membangun aplikasi monolitik *stateful* sebagai *baseline* sebelum melangkah ke arsitektur *stateless* dan *horizontal scaling*.

Sesuai dengan panduan penugasan, saya diminta untuk memenuhi beberapa ketentuan teknis:

- **Backend Monolitik & Single-Path Routing:** Menggunakan bahasa pemrograman murni tanpa *framework* besar (seperti Express-Generator atau Laravel). Seluruh alur data dan aksi dipusatkan pada satu titik masuk (*single URL Path*).
- **Manajemen State Server-Side:** Menggunakan *Server-Side Local Session* yang menyimpan data sesi langsung di memori RAM atau penyimpanan lokal server. Penggunaan JWT atau *token-based session* dilarang agar ketergantungan sesi pada satu mesin fisik dapat terlihat dengan jelas.
- **Basis Data Relasional:** Menerapkan RDBMS lokal (MariaDB/MySQL) di instance yang sama dengan hubungan *Foreign Key* dan opsi *cascading* antara tabel pengguna dan puisi.
- **Frontend Minimalis:** Menerapkan HTML, CSS, dan Fetch API murni tanpa *framework* UI untuk memantau mekanisme header Set-Cookie dan pengelolaan Session ID melalui Network tab browser.
- **Deployment AWS EC2:** Menjalankan aplikasi pada instance AWS EC2 (t2.micro) dan mengatur *Security Group Inbound Rules* agar dapat diakses oleh publik.

2. Perancangan Basis Data

Dalam perancangan sistem ini, saya membangun basis data relasional menggunakan MariaDB secara lokal di instance EC2 yang sama. Untuk menjaga integritas data, saya menerapkan relasi *Foreign Key* antara tabel users dan puisi dengan opsi *cascading* (ON DELETE CASCADE dan ON UPDATE CASCADE). Pendekatan ini saya pilih agar setiap perubahan atau penghapusan data pengguna dapat secara otomatis menyelaraskan data puisi yang terhubung. Skema basis data yang saya terapkan adalah sebagai berikut:

SQL

```
CREATE TABLE users (
```

```
id INT AUTO_INCREMENT PRIMARY KEY,  
username VARCHAR(50) NOT NULL UNIQUE,  
password VARCHAR(255) NOT NULL,  
nama VARCHAR(100) NOT NULL,  
no_id VARCHAR(50)  
);
```

```
CREATE TABLE puisi (  
id INT AUTO_INCREMENT PRIMARY KEY,  
user_id INT NOT NULL,  
judul VARCHAR(255) NOT NULL,  
tgl_submit DATE NOT NULL,  
isi TEXT NOT NULL,  
kategori VARCHAR(100),  
keyword VARCHAR(255),  
CONSTRAINT fk_puisi_user  
FOREIGN KEY (user_id)  
REFERENCES users(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

3. Arsitektur Backend dan Frontend

Untuk arsitektur aplikasi, saya mengimplementasikan *Single-Path Routing* menggunakan Node.js murni tanpa bergantung pada *framework* seperti Express-Generator. Seluruh logika aplikasi ditangani melalui satu titik masuk (*single entry point*) yang mengeksekusi empat aksi utama:

- register (POST) untuk pendaftaran akun pengguna baru.
- login (POST) untuk autentikasi sekaligus pembuatan sesi lokal di sisi server.
- submit_puisi (POST) untuk mengunggah konten puisi yang dilindungi oleh validasi sesi.
- daftar_puisi (GET) untuk mengambil dan menampilkan koleksi puisi pengguna yang sedang aktif.

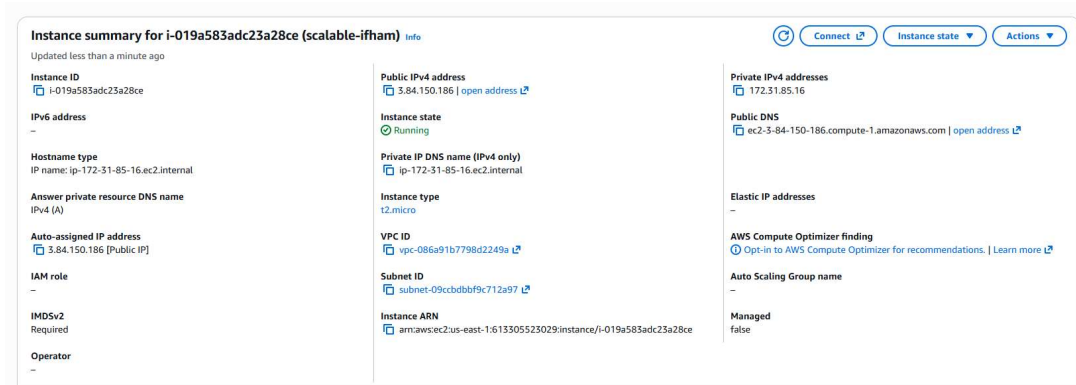
Di sisi antarmuka, saya membangun *frontend* minimalis menggunakan HTML, CSS, dan Fetch API murni tanpa *framework* UI seperti Bootstrap atau Tailwind. Pendekatan ini memudahkan saya untuk melakukan observasi *state* secara langsung melalui tab Network pada *browser*, terutama dalam mengamati bagaimana mekanisme header Set-Cookie bekerja untuk menjaga keabsahan Session ID pada setiap *request* yang terproteksi.

4. Deployment & Operasional Server (AWS EC2)

a. Inisialisasi Instance AWS EC2 Sandbox

Langkah awal deployment saya mulai di AWS Academy Sandbox dengan membuat instance EC2 baru.

- Saya membuat dan mengonfigurasi instance baru dengan nama `scalable-ifham` menggunakan AMI **Amazon Linux**.
- Setelah instance aktif, server mendapatkan alokasi **Public IP** (sebagai contoh percobaan pengujian: **3.84.150.186 / 44.211.47.205**) dan encryption RSA key yang nantinya digunakan untuk akses remote SSH dan pengujian aplikasi via *browser*.



b. Remote access

- Sebelum masuk ke server, saya mengamankan berkas *private key* di Windows Local Environment menggunakan PowerShell.

- Saya memindahkan berkas Kunci SSH ke folder terisolasi melalui perintah:
`Move-Item "C:\Users\Syafwan\Downloads\ifham-key.pem" "$HOME\.ssh\ifham-key.pem" -Force`
- Selanjutnya, saya memastikan berkas telah berpindah dengan perintah `Get-ChildItem "$HOME\.ssh"`.
- Setelah file siap, saya melakukan koneksi *remote* ke instance EC2 Amazon Linux menggunakan perintah: `ssh -i "$HOME\.ssh\ifham-key.pem" ec2-user@44.211.47.205`

```
PS C:\WINDOWS\system32> Move-Item "C:\Users\Syafwan\Downloads\ifham-key.pem" "$HOME\.ssh\ifham-key.pem" -Force
PS C:\WINDOWS\system32> ssh -i "$HOME\.ssh\ifham-key.pem" ec2-user@3.84.150.186
The authenticity of host '3.84.150.186 (3.84.150.186)' can't be established.
ED25519 key fingerprint is SHA256:ehnU/vQTxujf0p4pkaLV/DA4GtjOwR+BdWkevCQBTHM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.84.150.186' (ED25519) to the list of known hosts.

#_
##### Amazon Linux 2023
#####
#####|
#####|
#####|_#/_ https://aws.amazon.com/linux/amazon-linux-2023
#####|_#/_
```

c. Setup Environment Server

- Setelah berhasil masuk ke instance Amazon Linux 2023, saya menginstal seluruh *dependency* yang dibutuhkan.
- Saya mengunduh paket utama menggunakan paket manager dnf: `sudo dnf install -y nodejs npm git mariadb105-server tmux`
- Layanan basis data saya jalankan dan atur agar aktif otomatis saat *booting* dengan: `sudo systemctl start mariadb && sudo systemctl enable mariadb`
- Saya memastikan status MariaDB sudah berjalan normal melalui `sudo systemctl status mariadb`.

d. Konfigurasi Basis Data & Impor Skema

- Saya masuk ke prompt MariaDB sebagai root menggunakan `sudo mariadb`.
- Di dalamnya, saya membuat basis data baru lewat perintah `CREATE DATABASE scalable_one;`
- Saya membuat pengguna khusus agar tidak menggunakan akun root: `CREATE USER 'scalable'@'localhost' IDENTIFIED BY 'Ifhamfkr050706';`
- Hak akses penuh saya berikan melalui `GRANT ALL PRIVILEGES ON scalable_one.* TO 'scalable'@'localhost'; FLUSH PRIVILEGES;` lalu keluar dari prompt.
- Terakhir, saya mengimpor skema tabel yang sudah disiapkan di file DDL ke basis data dengan perintah:

- `mysql -u scalable -p scalable_one < schema.sql`

e. Deployment Aplikasi Node.js & Pengelolaan Background Process

Untuk menjalankan aplikasi di server produksi, saya menarik kode sumber dan mengonfigurasi variabel lingkungan.

- Saya mengkloning repositori proyek dari GitHub menggunakan `git clone` <https://github.com/ifham-ux/scalable-test1.git> lalu masuk ke direktori tersebut.
- Saya menginstal seluruh modul dependensi Node.js dengan `npm install`.
- Saya membuat berkas `.env` (`touch .env`) untuk menyimpan kredensial basis data dan port secara aman:

None

```
DB_HOST=localhost
DB_USER=scalable
DB_PASSWORD=Ifhamfkr050706
DB_NAME=scalable_one
DB_PORT=3306
PORT=8080
```

- Agar aplikasi tetap berjalan di latar belakang meskipun koneksi SSH saya tutup, saya memanfaatkan `tmux`. Saya membuka sesi baru dengan perintah `tmux`, lalu menjalankan server menggunakan `node server.js`.

```
[ec2-user@ip-172-31-85-16 scalable-test1]$ node server.js
> injected env (6) from .env // tip: ♦ secrets for agents [www.dotenvx.com]
Server running at on port 8080
```

- Saya memutus tampilan terminal dari sesi tersebut tanpa menghentikan aplikasinya dengan menekan kombinasi tombol `Ctrl+B` lalu `D` (*Detach*).
- Apabila ada pembaruan kode di kemudian hari, saya cukup menjalankan `git pull` <https://github.com/ifham-ux/scalable-test1.git> untuk menyelaraskannya.

f. Pengujian Inbound Rules & Akses Publik

Langkah terakhir yang saya lakukan adalah mengonfigurasi *Inbound Rules* pada *Security Group* instance AWS EC2 (`scalable-ifham`) untuk mengidentifikasi dan mengizinkan lalu lintas masuk (*incoming traffic*) spesifik pada port HTTP `8080`. Setelah aturan jaringan dibuka, saya menguji aplikasi langsung melalui *browser* di

alamat <http://3.84.150.186:8080> dan memastikan seluruh alur fungsi berjalan dengan baik.

5. Deliverables Penugasan

Item Deliverable	Detail / Tautan
Repositori GitHub	github.com/ifham-ux/scalable-test1
AWS EC2 Public IP	3.84.150.186 / 44.211.47.205
Akses Publik Browser	http://3.84.150.186:8080
Status Deployment	Berjalan Aktif (<i>Stateful Session via Tmux</i>)

6. Lampiran

None

```
ssh -i "$HOME\.ssh\ifham-key.pem" ec2-user@44.211.47.205

Move-Item "C:\Users\Syafwan\Downloads\ifham-key.pem" "$HOME\.ssh\ifham-key.pem"
-Force

Remove-Item "$HOME\.ssh\ifham-key.pem"

Get-ChildItem "$HOME\.ssh"

sudo dnf install -y nodejs npm

sudo dnf install git -y

sudo dnf install mariadb105-server -y

sudo dnf install tmux -y

sudo systemctl start mariadb

sudo systemctl enable mariadb

sudo systemctl status mariadb

git clone https://github.com/ifham-ux/scalable-test1.git

sudo mariadb

mysql -u scalable -p scalable_one < schema.sql

CREATE USER 'scalable'@'localhost'

IDENTIFIED BY 'Ifhamfkr050706';

GRANT ALL PRIVILEGES ON scalable_one.*

TO 'scalable'@'localhost';

FLUSH PRIVILEGES;

exit;

mysql -u scalable -p
```

```
create database scalable_one;

mysql -u scalable -p scalable_one < schema.sql

npm install

creating .env file in directory

tmux then ctrl B and D to attach them running on background on

run node server.js

git pull https://github.com/ifham-ux/scalable-test1.git

running this on browser http://3.84.150.186:8080
```

The image shows a registration form titled "Register" centered on a background with a green-to-yellow gradient and a light gray dot grid. The form is a white rounded rectangle containing three input fields labeled "Full Name", "Username", and "Password". Below these fields is a dark gray button with the text "Register" in white. At the bottom of the form, it says "Have an account? [Login](#)".

Dashboard

Submission Form

Title

Content

Category

Keywords

Submit Puisi

Submitted Poem

2026-08-31T17:00:00.000Z

lasksdffja

adffidffj

2026-08-31T17:00:00.000Z

asdf

asdfs

2026-08-31T17:00:00.000Z

ads

saf

2026-08-31T17:00:00.000Z

afdas